

Introduction to Computer Organisation and Architecture UCCD1133



Chapter 6

Processor, Memory System and Instruction Execution

Disclaimer



- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

CH 6.2:

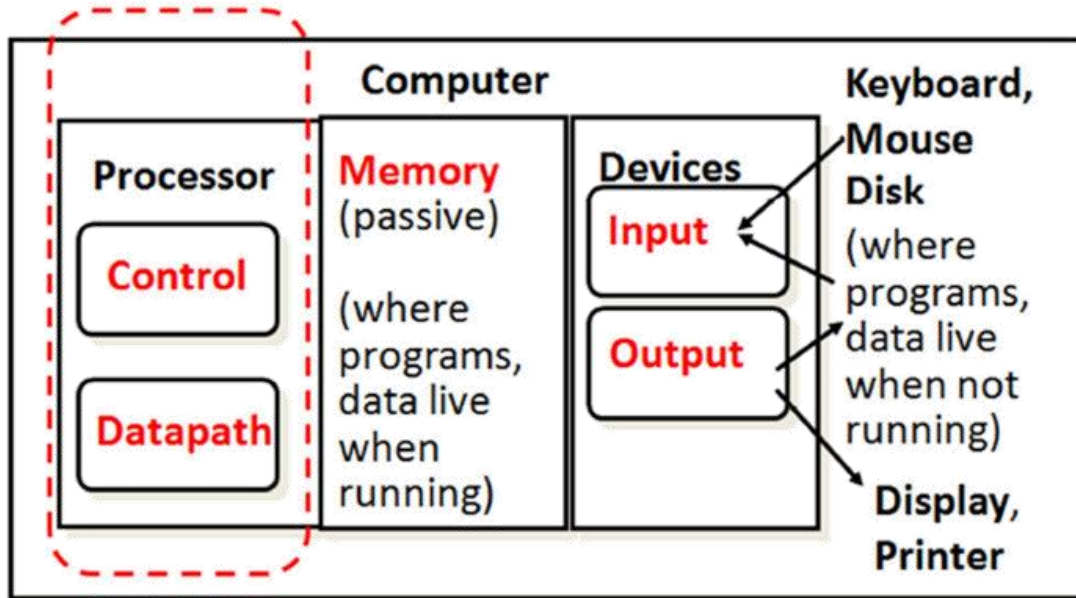
Processors: Single-cycle, Multi-cycle and Pipeline

Microarchitecture:

- Single-cycle Processor
- Multicycle Processor
- Pipelined Processor

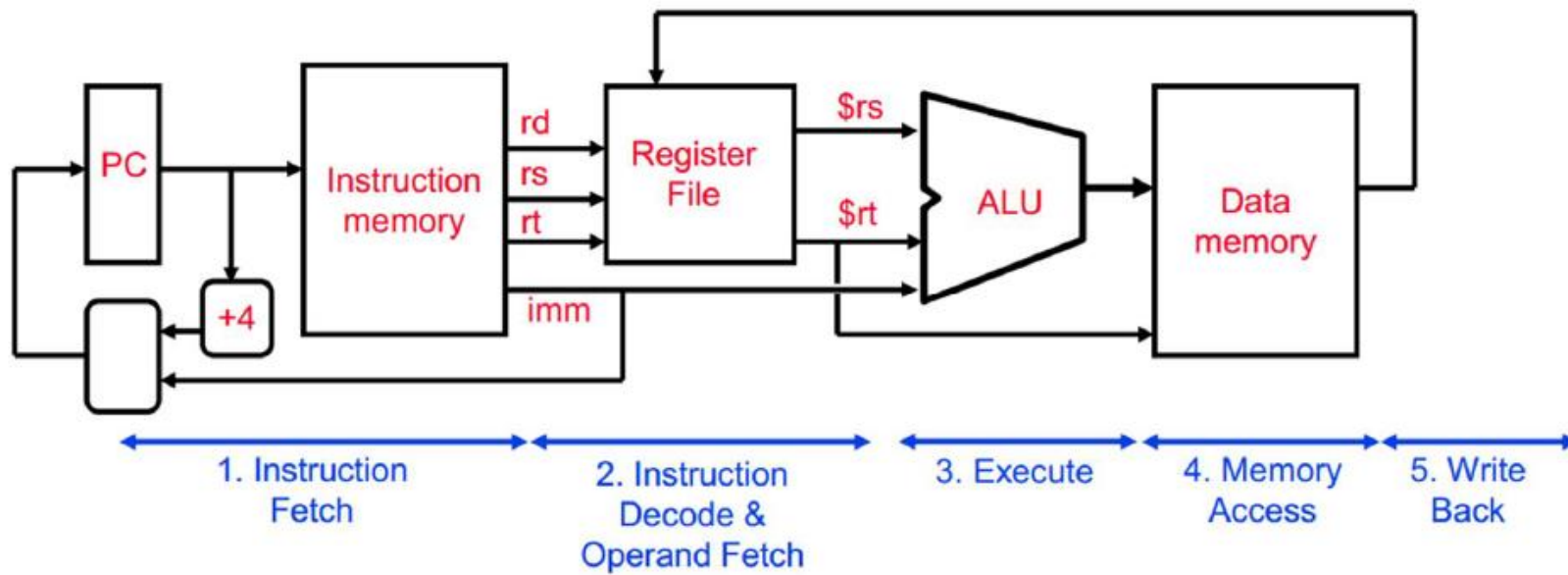
Microarchitecture

- **Microarchitecture:** how to implement an architecture in hardware
- Processor:
 - **Datapath:** functional blocks
 - **Control:** control signals



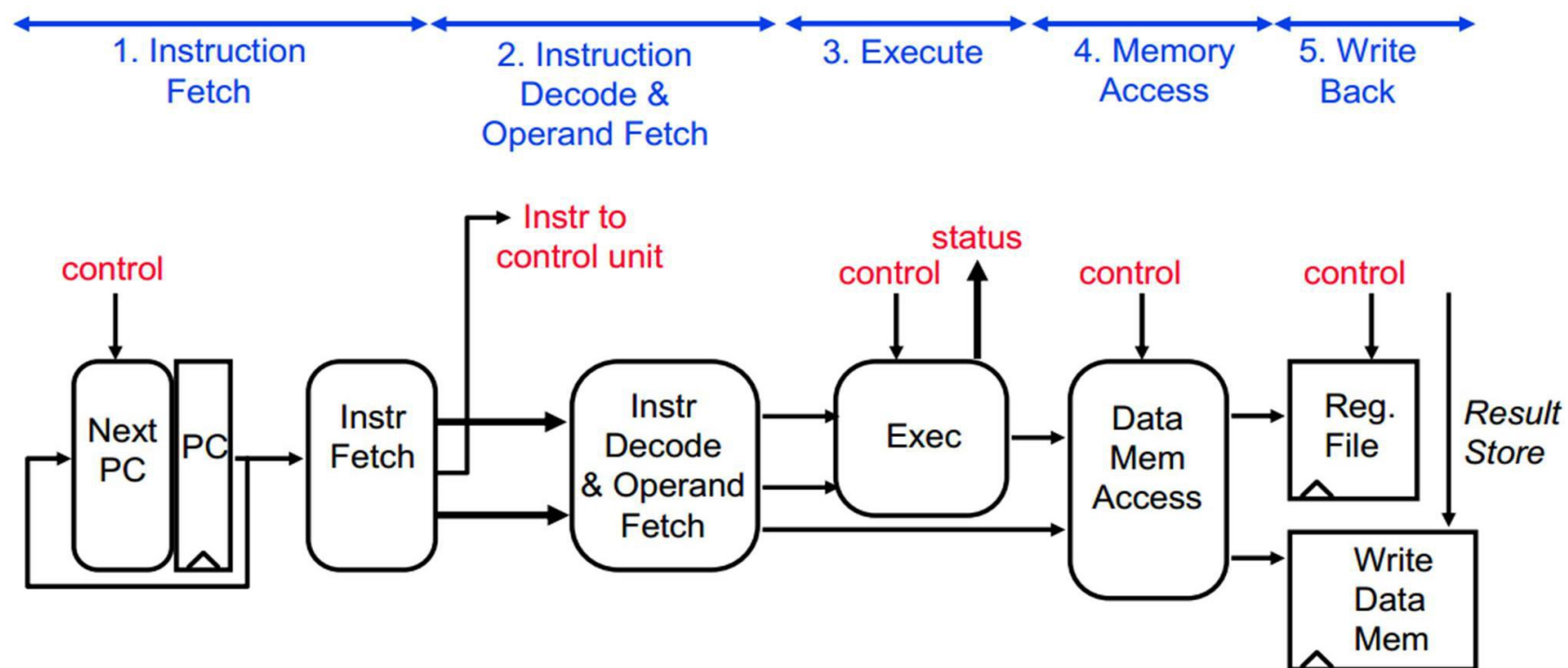
Datapath Abstract Implementation View

- Assemble the components
 - Based on stages of “instruction execution cycle”
 - Obtain an abstract datapath implementation view.
 - Note : one could have different number of stages for different CPU architecture



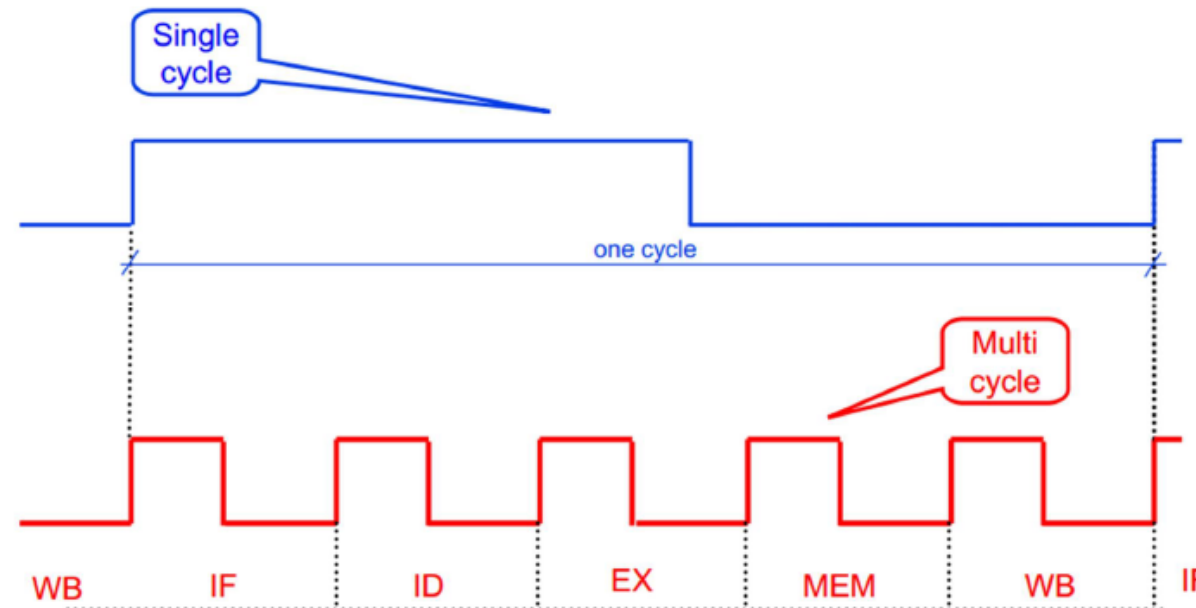
Datapath Abstract Implementation View

- Datapath with control signals:



Microarchitecture

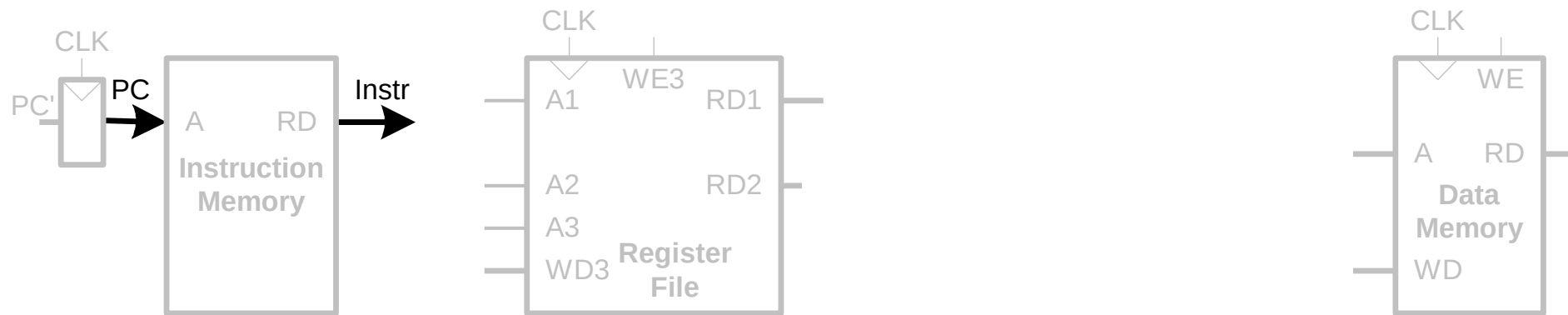
- Multiple implementations for a single architecture:
 - **Single-cycle:** Each instruction executes in a single cycle
 - **Multicycle:** Each instruction is broken into series of shorter steps
 - **Pipelined:** Each instruction broken up into series of steps & multiple instructions execute at once



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

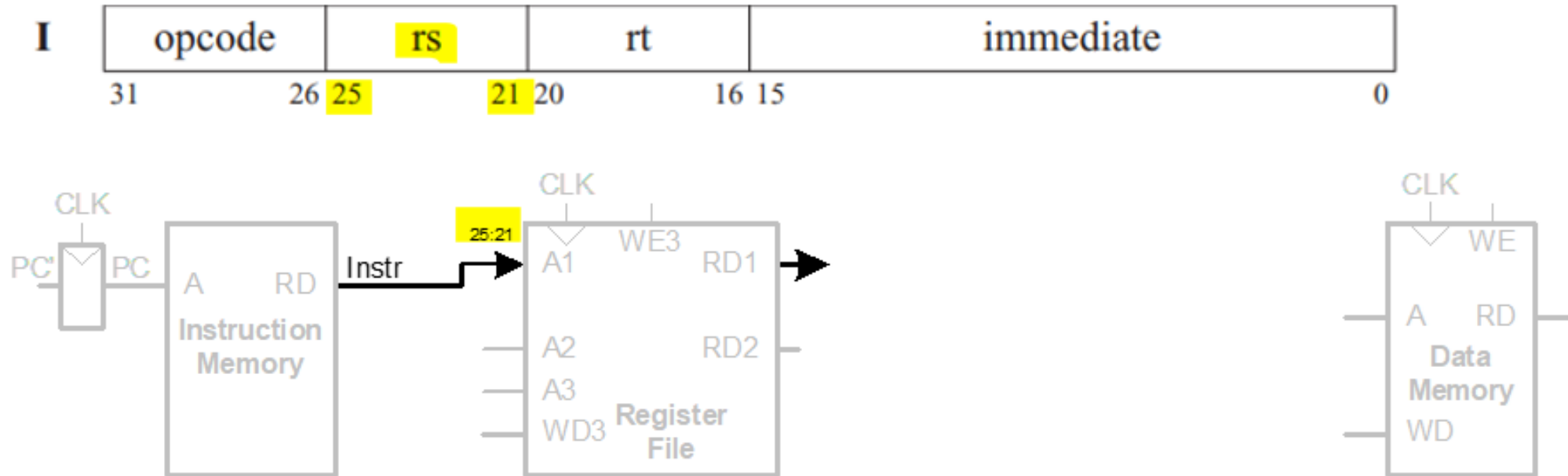
STEP 1: Fetch instruction – fetch the instruction from the instruction memory



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

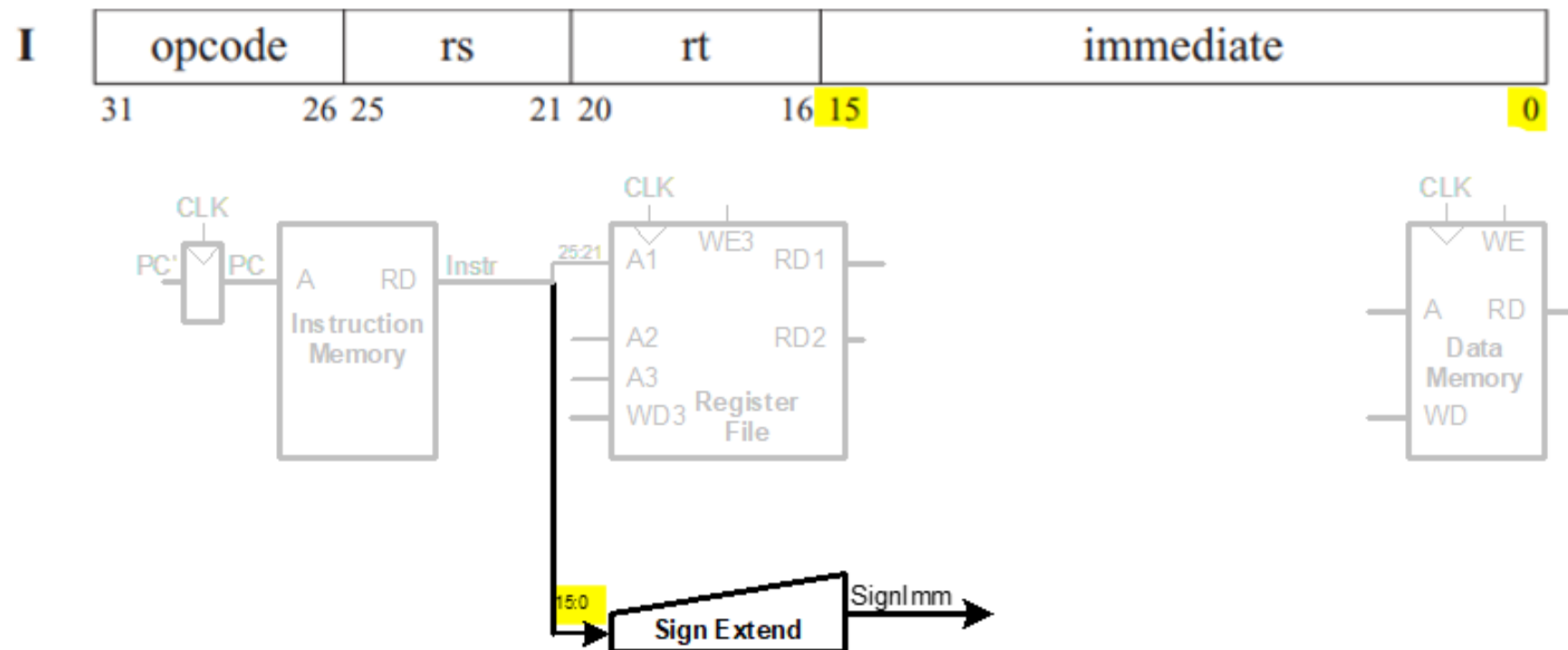
STEP 2: Read source operands from register file



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

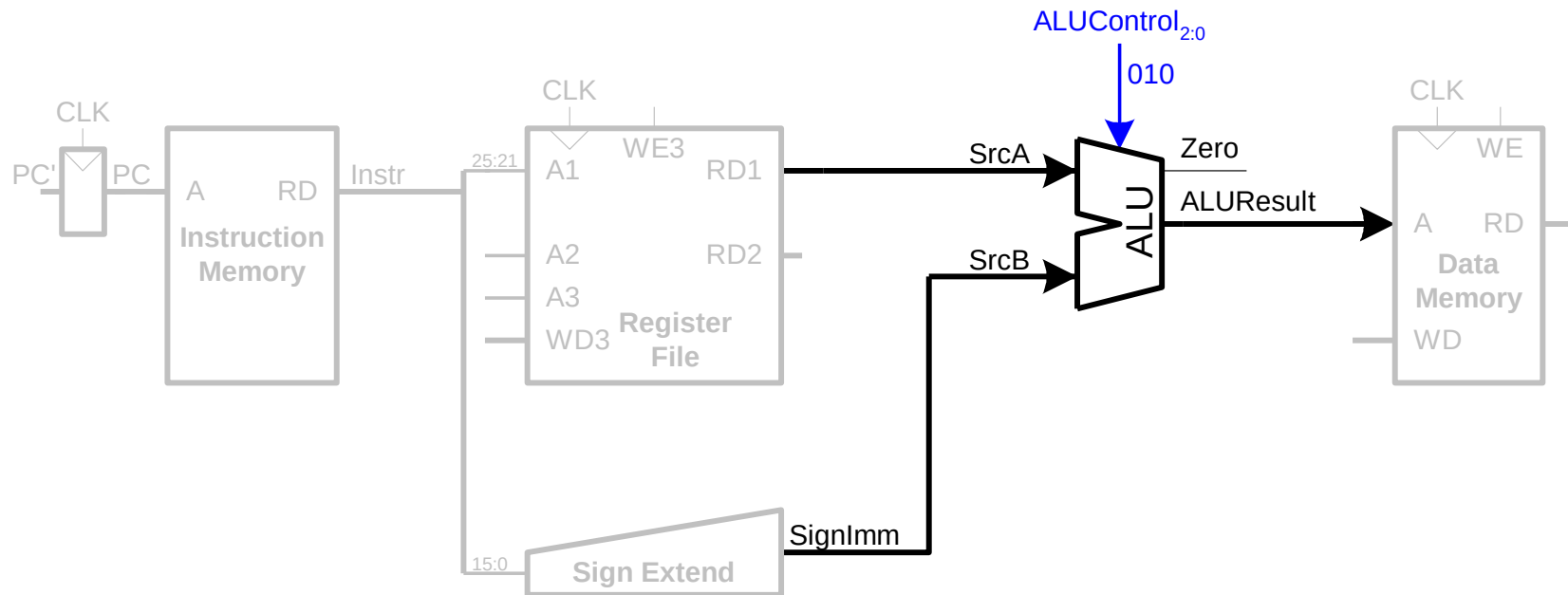
STEP 3: Sign-extend the immediate



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

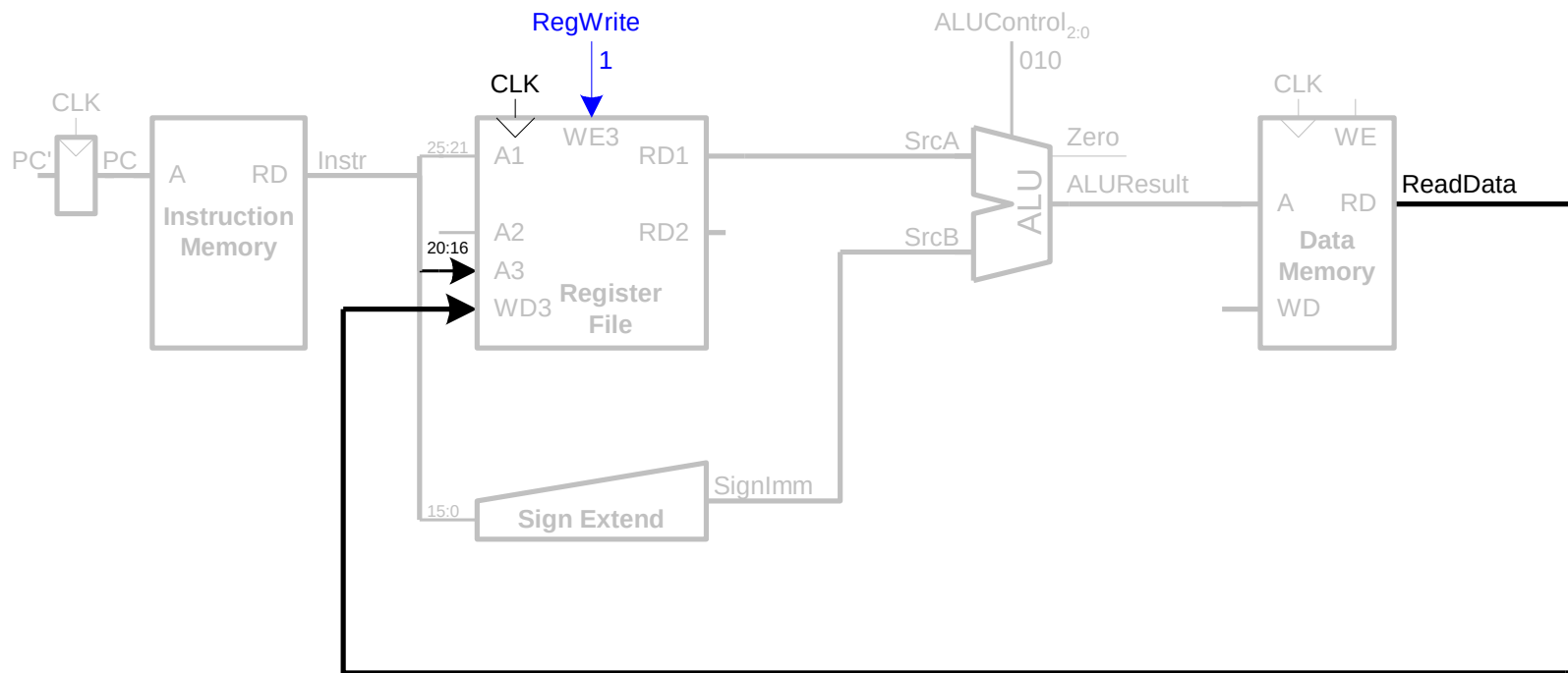
STEP 4: Compute the memory address



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

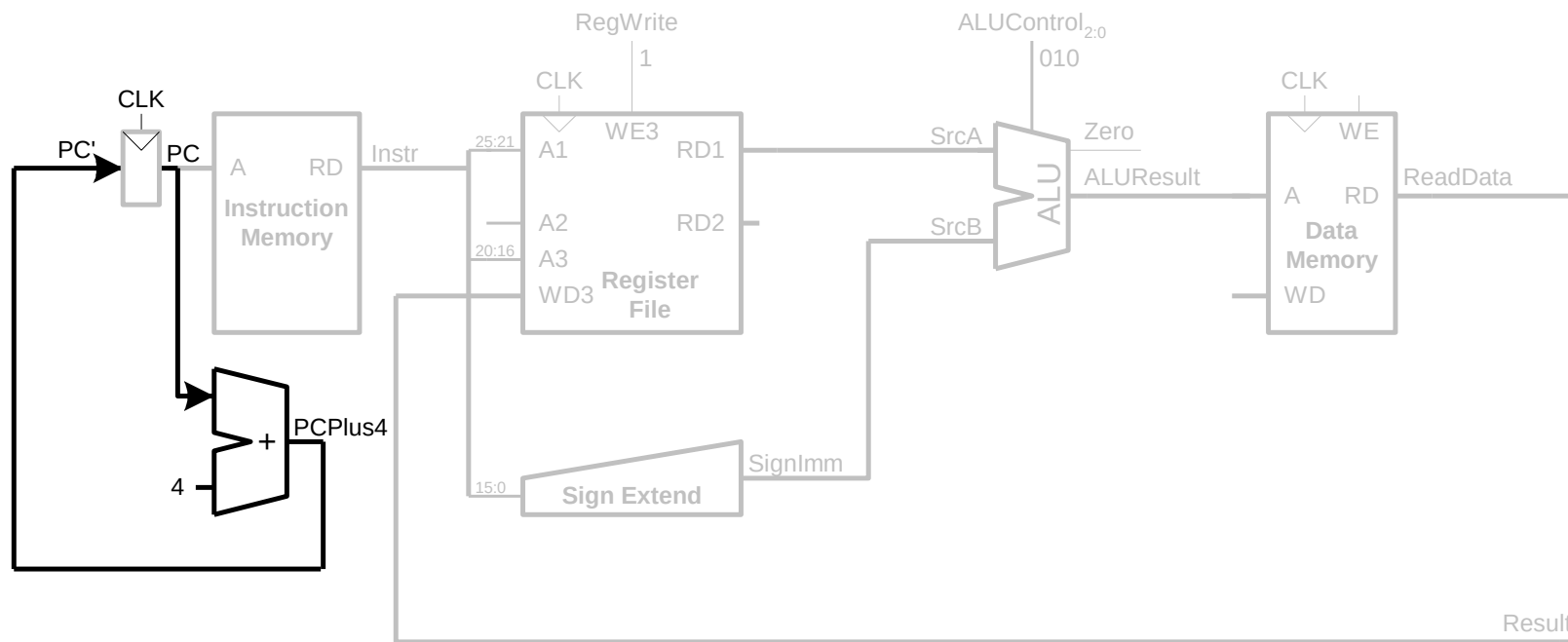
STEP 5: Read data from memory and write it back to register file



Single-Cycle Datapath:

Example: `lw rt, imm(rs)`

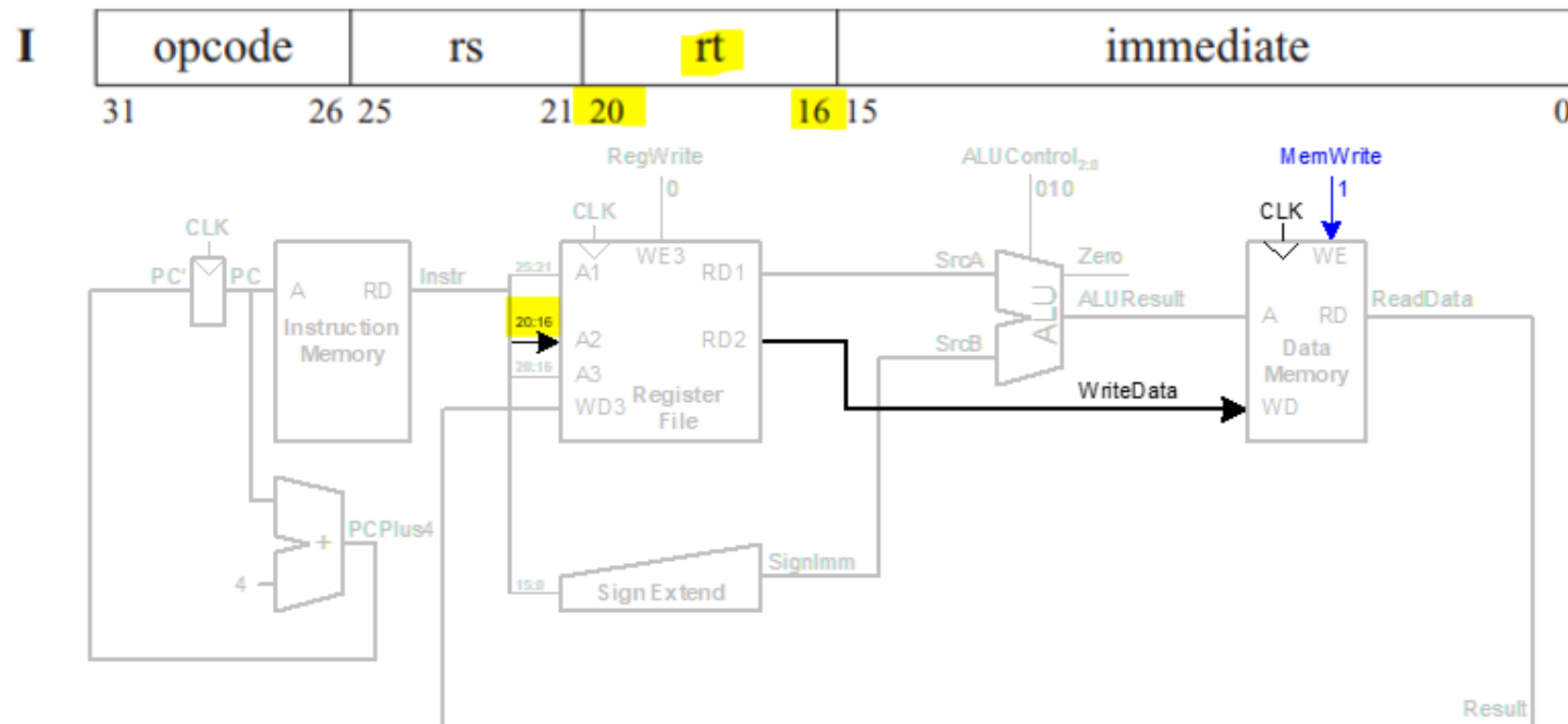
STEP 6: Determine the address of the next instruction



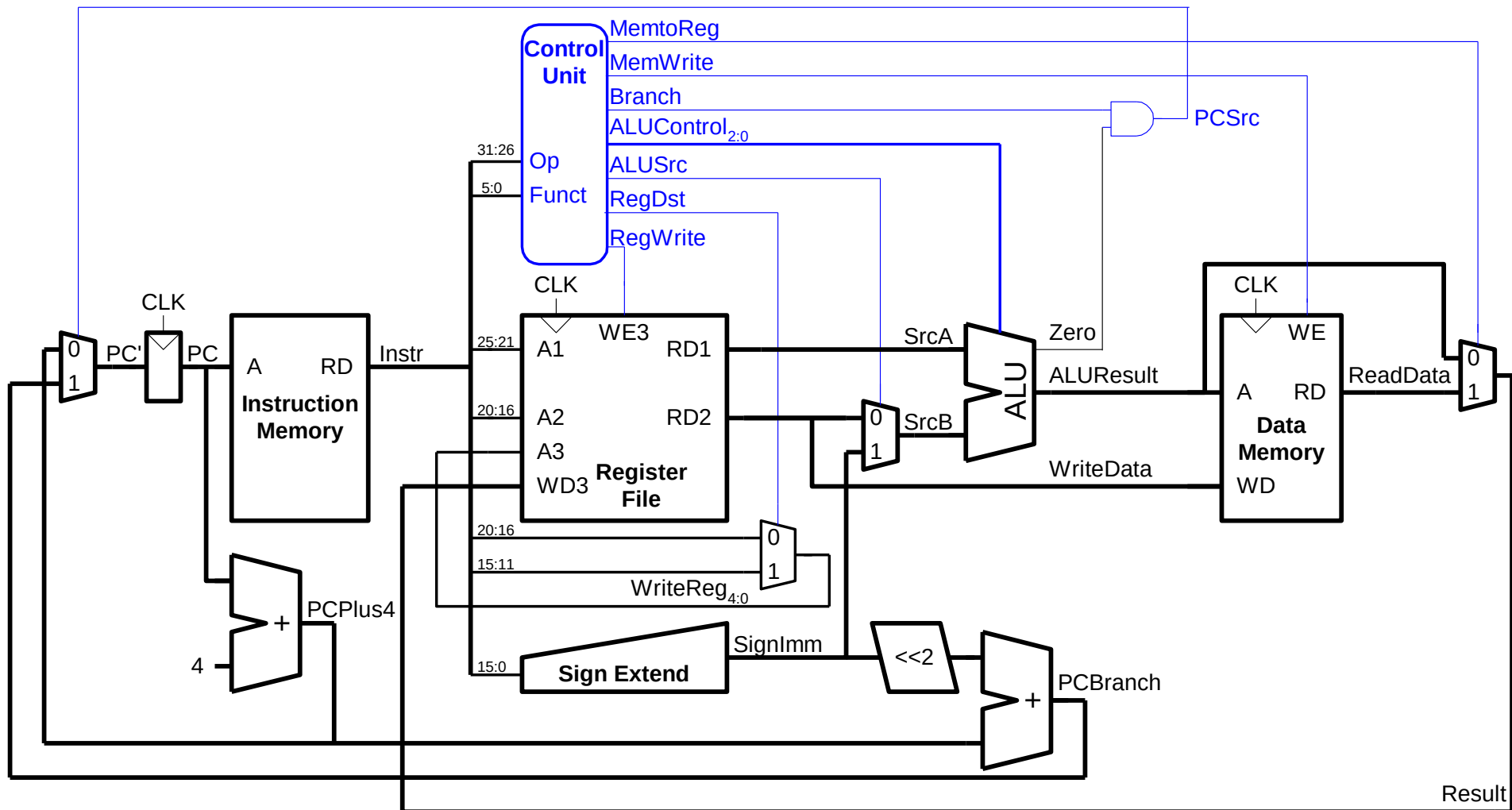
Single-Cycle Datapath:

Example: `sw rt, imm(rs)`

Write data in rt to memory

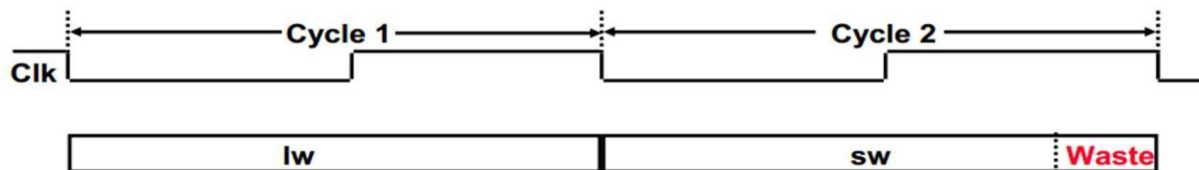


Single-Cycle Processor



Drawback of Single-Cycle

- Uses the **clock cycle inefficiency** – the clock cycle is set to accommodate the **slowest instruction**, and must be the **same length** for all instructions
 - Problem:
 - all instructions take as much time as the slowest instruction than needed
 - Especially problematic for more complex instructions like floating point multiply
- Need to **duplicate resources** that are used more than once per cycle – waste area and power.
 - Some functional units (eg. adders) must be duplicated since they can not be shared during a clock cycle
- Real memory is slower than idealized memory
 - Cannot always get the job done in one (short) cycle
 - Need to further stretch the clock period – slower.

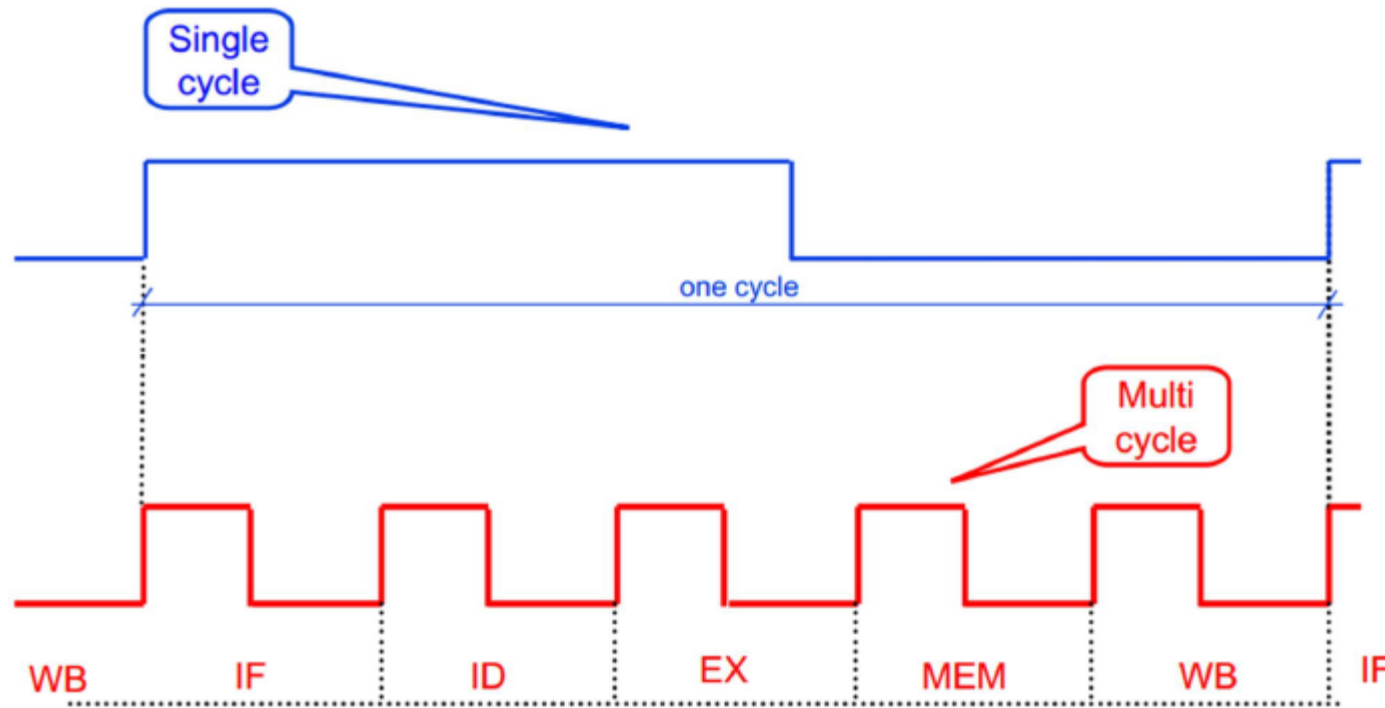


Multicycle MIPS Processor

- Single-cycle datapath => **CPI=1, CCT => long**
- Multi-cycle processors
 - Require **more complex control** – control is the hard part.
 - **Avoid idling** – different instructions to take a different number of clock cycles.
 - **Faster clock rates.**
 - Data path allows **greater sharing of hardware.**
- MIPS makes control easier
 - Instructions same size
 - Source registers always in same place
 - Immediate same size, location
 - Operations always on registers/immediates
- Functional units can be used more than once per instruction as long as they are used on different clock cycles, as a result
 - **Only one memory** – holding both instructions and data
 - **Only one ALU** (used almost every cycle) – saves two adders

Multicycle MIPS Processor

One Cycle to Multi-Cycle Conversion



Single cycle:

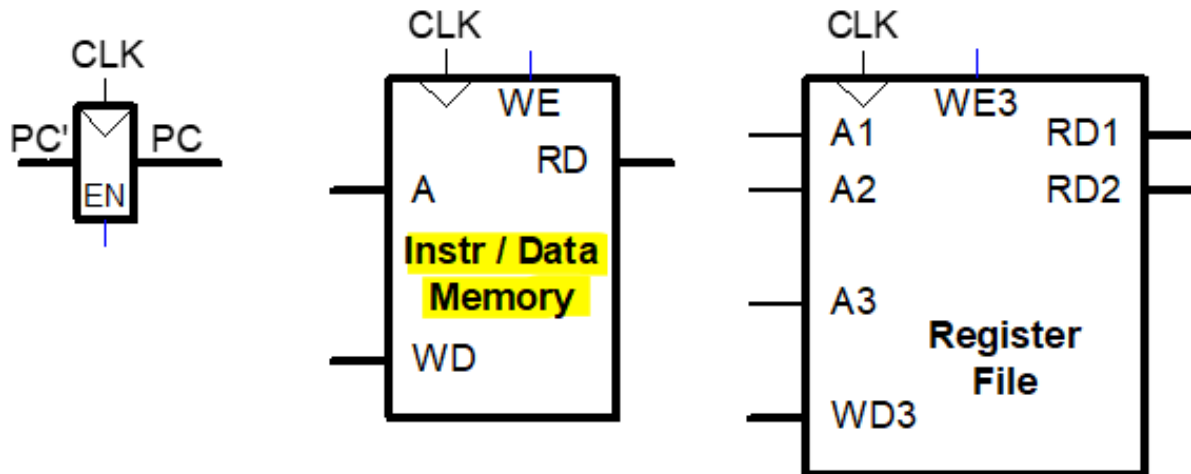
- Each instruction is executed within a single clock cycle
- Clock cycle time set to accommodate the slowest instruction

Multicycle:

- Instructions are divided into multiple stages
- Each stage takes one or more clock cycles to complete
- Different instructions may require different number of cycles

Multicycle State Elements

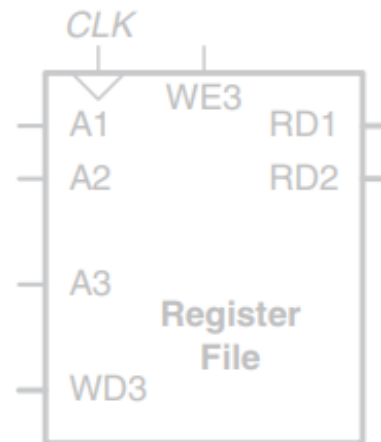
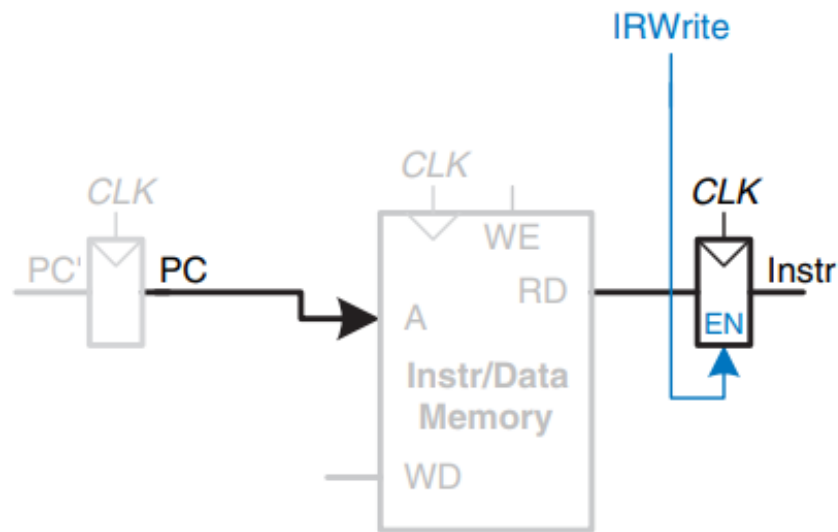
- Replace Instruction and Data memories with a **single unified memory** – more realistic



Multicycle Datapath:

Example: `lw rt, imm(rs)`

STEP 1: Fetch instruction

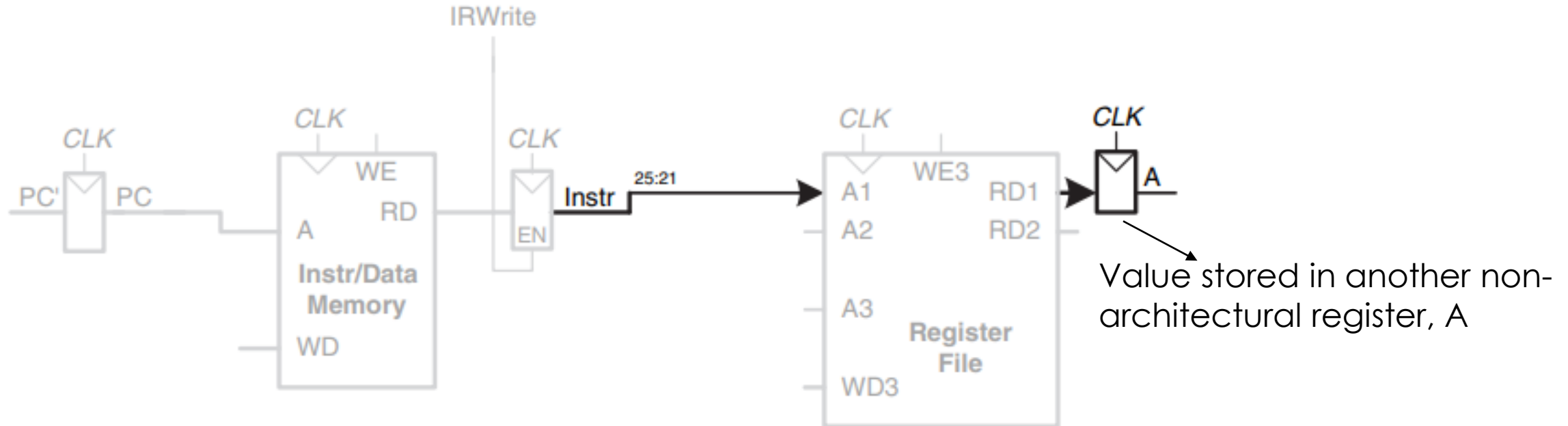


- The PC is connected to the address input of the instruction memory
- The instruction is read and stored in a new non-architectural **Instruction Register** so that it is available for future cycles

Multicycle Datapath:

Example: `lw rt, imm(rs)`

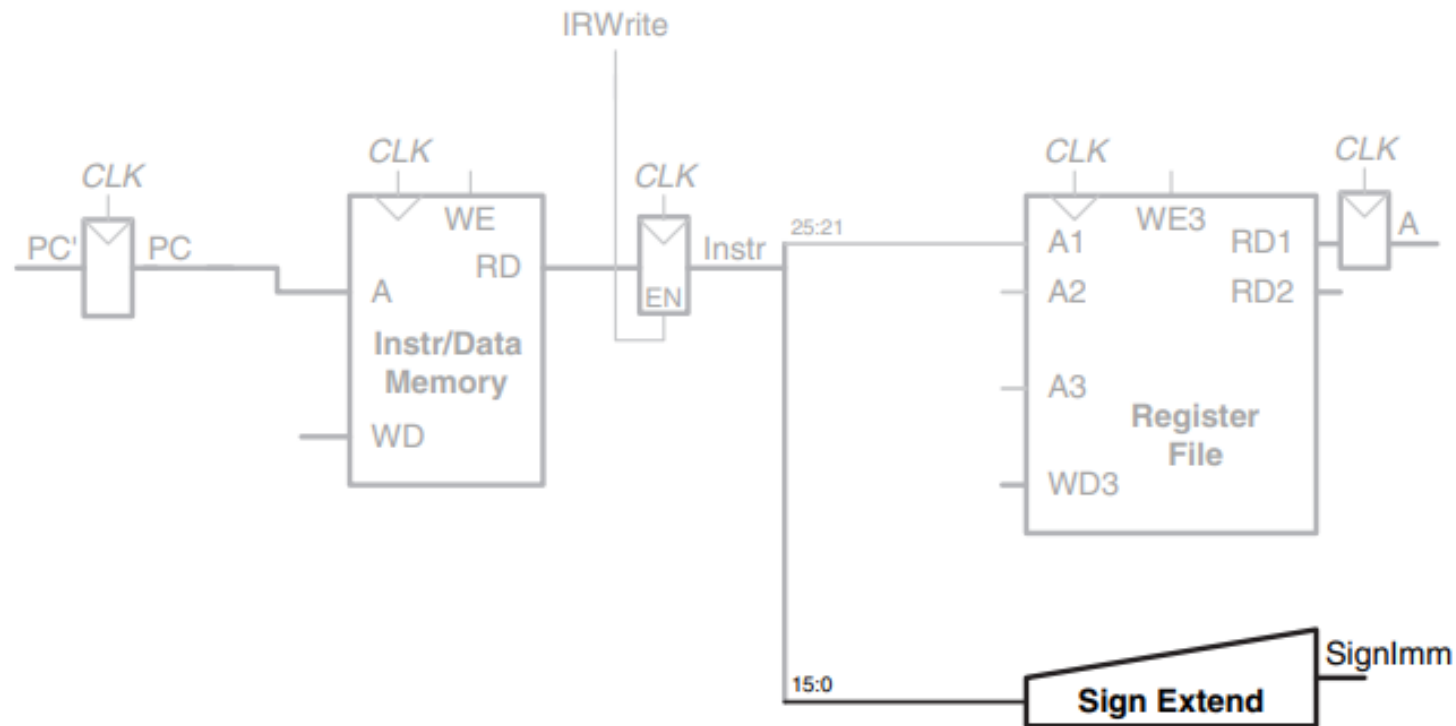
STEP 2a: Read source operands from register file



Multicycle Datapath:

Example: `lw rt, imm(rs)`

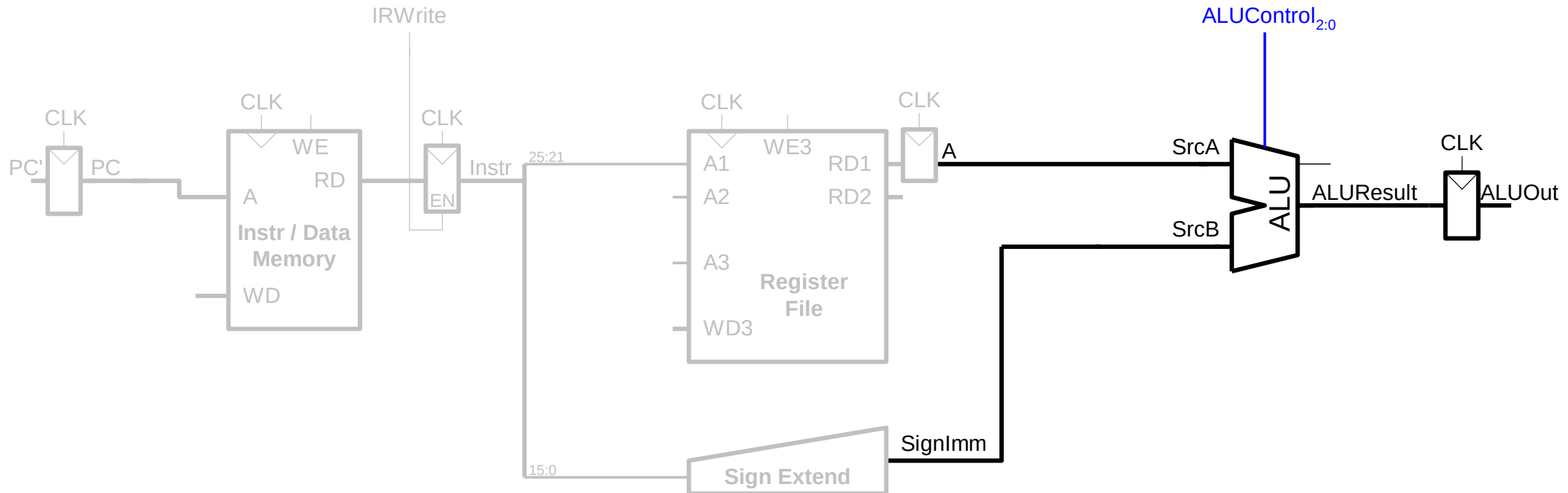
STEP 2b: Sign-extend the immediate



Multicycle Datapath:

Example: `lw rt, imm(rs)`

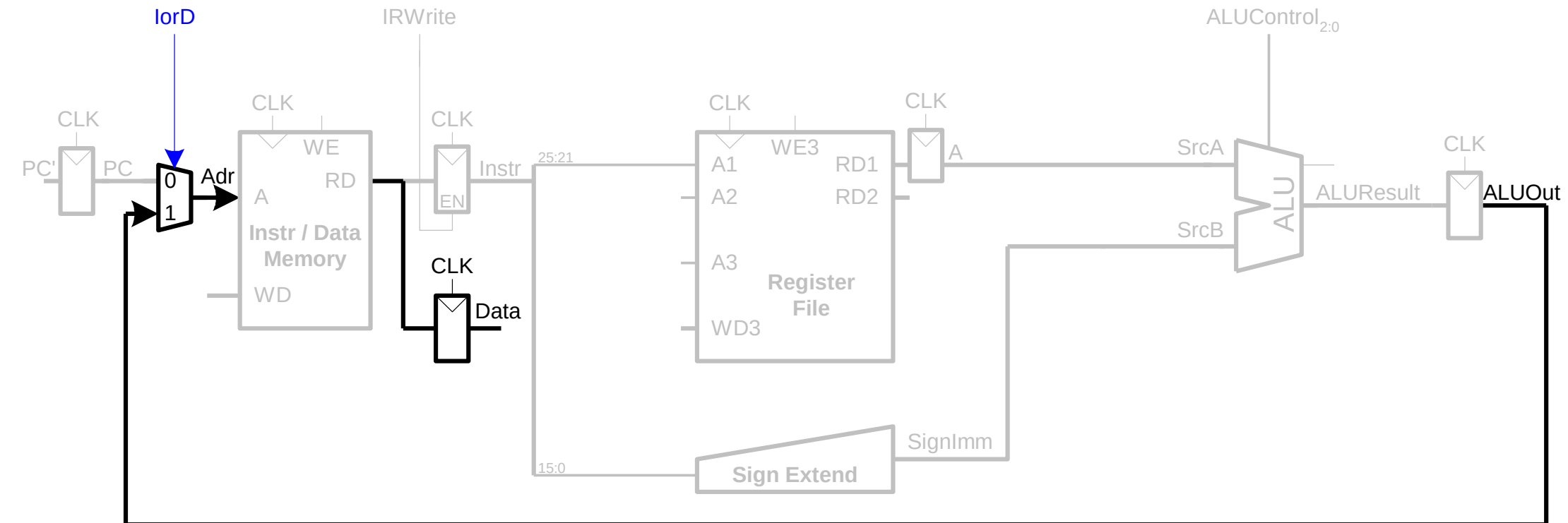
STEP 3: Compute the memory address



Multicycle Datapath:

Example: `lw rt, imm(rs)`

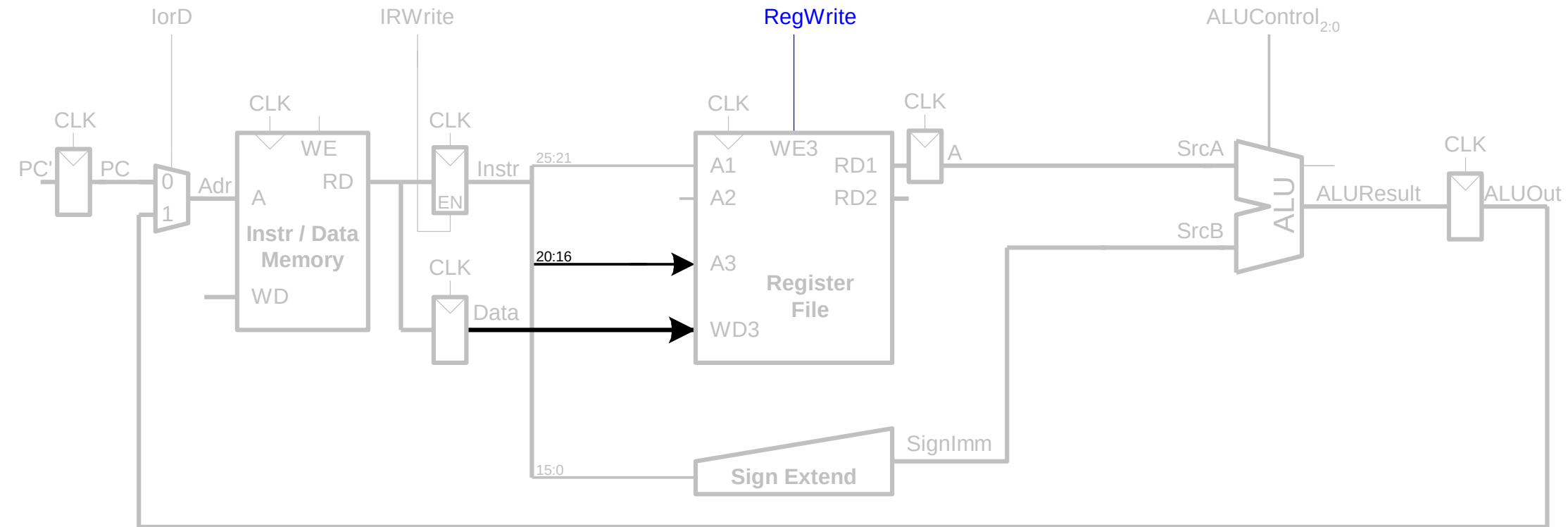
STEP 4: Read data from memory



Multicycle Datapath:

Example: `lw rt, imm(rs)`

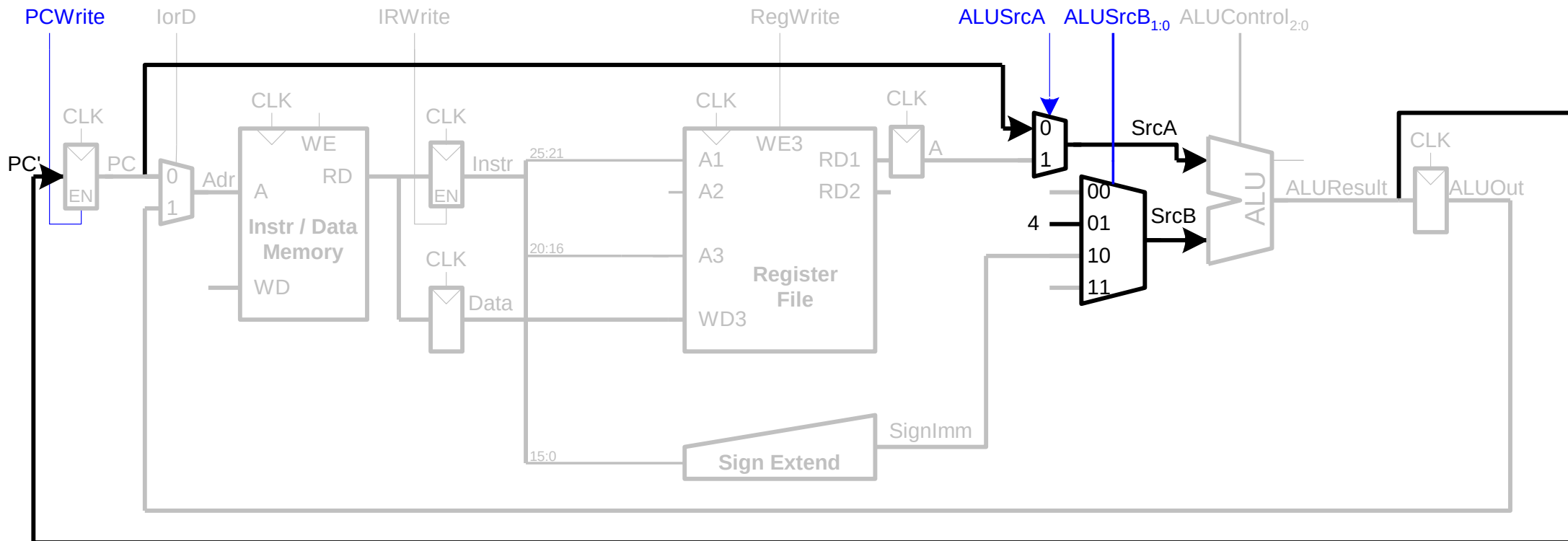
STEP 5: Write data back to register file



Multicycle Datapath:

Example: `lw rt, imm(rs)`

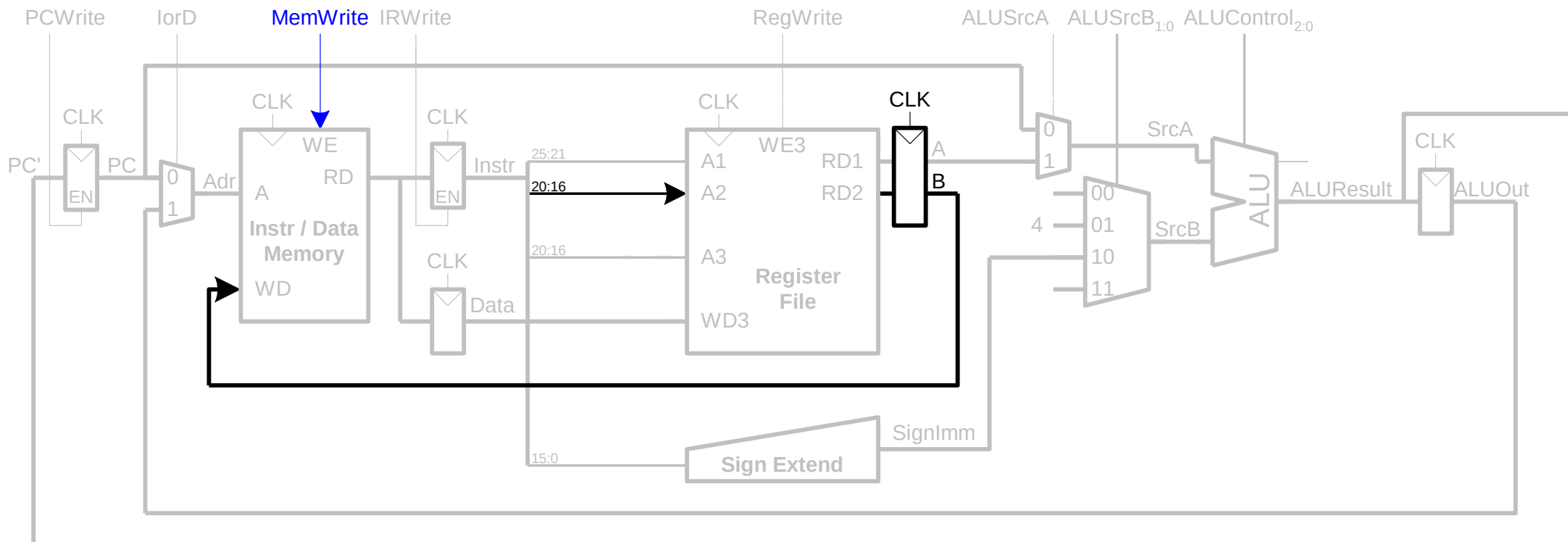
STEP 6: Increment of PC



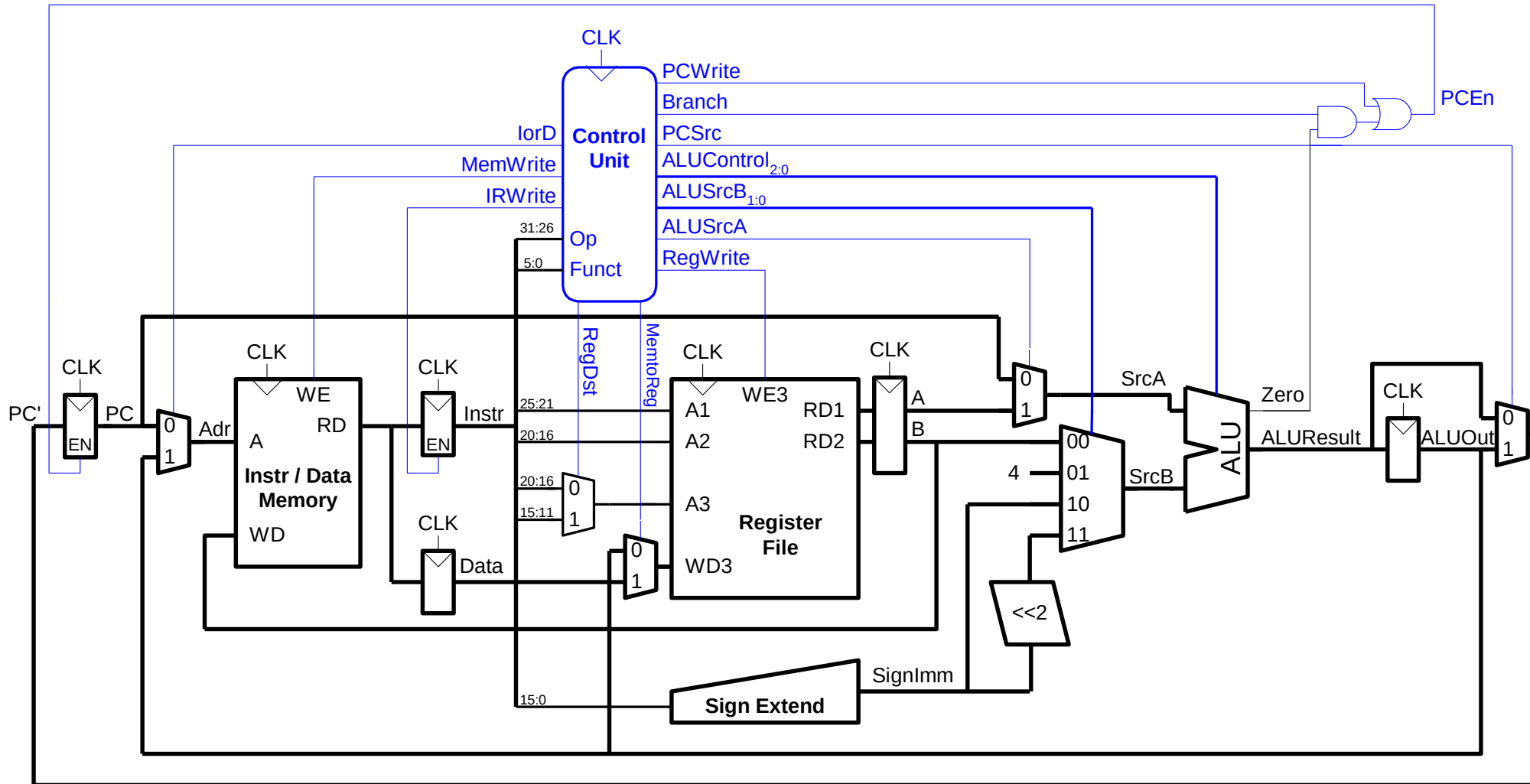
Multicycle Datapath:

Example: `sw rt, imm(rs)`

Write data in rt to memory

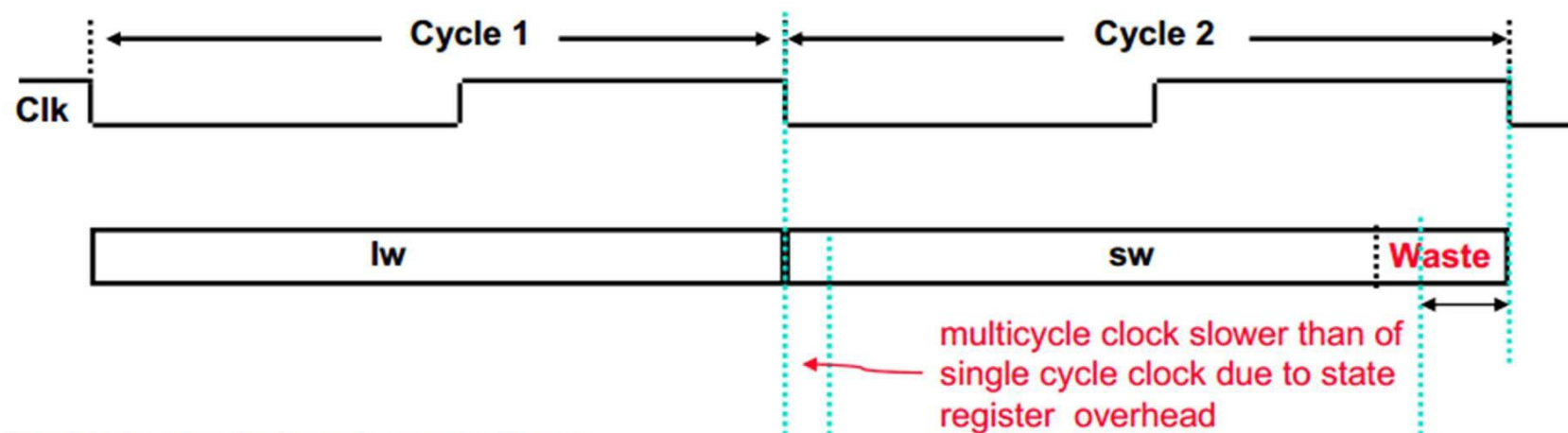


Multicycle Processor

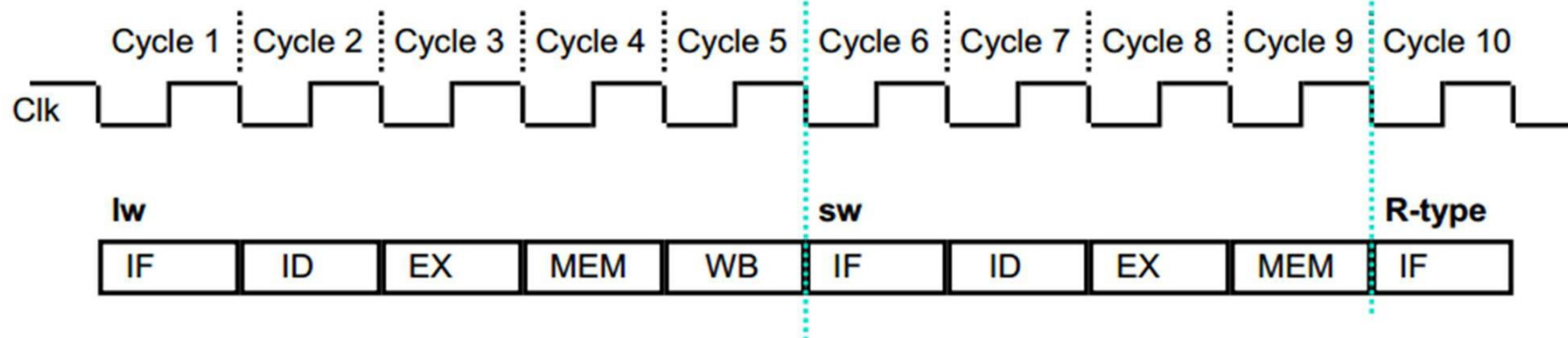


Single-Cycle vs Multicycle Timing

Single Cycle Implementation:



Multiple Cycle Implementation:





Q & A
Thank you